Конспект лекций

Архитектура ЭВМ и Ассемблер

к.ф.-м.н. Филонов П. В.

2013 г. / Редакция 2026 г.

Аннотация

Данный конспект охватывает фундаментальные принципы построения вычислительных систем, начиная от архитектуры фон Неймана и заканчивая практическими навыками программирования на языке ассемблера x86 (NASM).

Содержание

1 Лекция 1. Многопроцессорные системы **2**
1.1 Архитектура Джона фон Неймана 2
1.2 Пути увеличения производительности 2
1.2.1 Параллелизм уровня инструкций (ILP) 2

2 Лекция 2. Двоичная арифметика **2**
2.1 Дополнительный код 2

3 Практика: Программирование **2**
3.1 Учебный процессор GaLyA 2
3.2 Ассемблер x86 (NASM) 3

1 Лекция 1. Многопроцессорные системы

1.1 Архитектура Джона фон Неймана

Классическая модель базируется на четырёх принципах. Одной из ключевых проблем является *«узкое место»* — ограничение пропускной способности между процессором и памятью.

- **Принцип однородности памяти:** Команды и данные неразличимы.
- **Принцип адресности:** Доступ по уникальным номерам ячеек.
- **Принцип программного управления:** Последовательное выполнение.
- **Принцип двоичного кодирования:** Использование алфавита $\{0, 1\}$.

1.2 Пути увеличения производительности

1.2.1 Параллелизм уровня инструкций (ILP)

Для ускорения работы современных CPU используются:

- **Конвейеризация:** Разделение выполнения команды на стадии.
- **Суперскалярность:** Несколько исполнительных блоков на ядро.
- **SMT (Hyper-Threading):** Логическое разделение ресурсов ядра.

2 Лекция 2. Двоичная арифметика

2.1 Дополнительный код

Для представления отрицательных чисел в ЭВМ используется алгоритм:

1. Инверсия бит числа ($x \to \bar{x}$).
2. Прибавление единицы ($\bar{x} + 1$).

Размер	Беззнаковые	Знаковые
1 байт	0 .. 255	-128 .. 127
2 байта	0 .. 65535	-32768 .. 32767
4 байта	0 .. $2^{32} - 1$	$-2^{31}..2^{31} - 1$

3 Практика: Программирование

3.1 Учебный процессор GaLyA

Пример программы для поиска НОД (Алгоритм Евклида):

```
;                                   (a, b)
a:      db 0x0F    ; 15
b:      db 0x0A    ; 10

start:
        R0 := RAM[a]
        R1 := RAM[b]
loop:
        R2 := R0
        R2 := R2 mod R1    ; R2 = R0 % R1
        R0 := R1
        R1 := R2
        JNZ loop           ;           R1 != 0,
        HLT                ;                          R0
```

3.2 Ассемблер x86 (NASM)

В отличие от GaLyA, в x86 умножение и деление используют фиксированные пары регистров.

Размер	Множитель	Результат	Делимое
8 бит	AL	AX	AX
16 бит	AX	DX:AX	DX:AX
32 бита	EAX	EDX:EAX	EDX:EAX

Пример логического сдвига для быстрого умножения на 4:

```
mov eax, [val]
shl eax, 2       ; EAX = EAX * 4
```